

From Graph-Structured Lambda Theories to Rholang: Compiling Set-Automaton Interpreters with Inherent Concurrency

May 14, 2026

Abstract

A finite presentation of a graph-structured lambda theory (GSLT) specifies a small-step operational semantics by a finite signature of term constructors, a finite set of equations, and a finite inductive description of a reduction relation. We give a complete algorithm that, from such a presentation, produces a Rholang program implementing the corresponding small-step interpreter. The interpreter is structured as a Bouwman–Erkens–Groote *set automaton* for pattern matching against the left-hand sides of the presentation, paired with a fixed schedule of substitution and rewrite-rule firing. Each state of the set automaton is reflected to a Rholang channel; each transition is realised by a persistent **for**-receive on that channel; the indeterminate order of processing position pointers is realised by the indeterminate order of message delivery on those channels. We prove a *parallelism preservation* theorem: the asynchronous, name-passing semantics of Rholang makes every parallelism opportunity exposed by the set automaton — both intra-term (independent position pointers) and inter-term (independent redex firings) — directly available to the Rholang runtime without further scheduling glue. The compiler is total on any finitely presented GSLT, the produced program is of size polynomial in the automaton, and the resulting interpreter inspects each function symbol of the subject term exactly once per redex search.

Contents

1	Introduction	2
2	Preliminaries	3
2.1	Graph-structured lambda theories	3
2.2	The set automaton	5
2.3	Rholang	7
3	Encoding terms and states	8
3.1	Subject terms	8
3.2	Automaton states as channels	9
3.3	Position pointers	9
4	The compilation algorithm	9
4.1	Preamble	10
4.2	Term constructors	10
4.3	Traversal contract	11

4.4	State contracts	11
4.5	Equational normaliser	12
4.6	Rewriter	13
4.7	Driver	14
5	Parallelism preservation	15
5.1	Concurrency model	15
5.2	Intra-state parallelism	15
5.3	Inter-rule parallelism	15
5.4	Why this matters	16
6	A worked example	16
6.1	The presentation	16
6.2	The set automaton	16
6.3	The compiled Rholang program	17
6.4	Running it	19
6.5	The rho COMM rule: full compilation pipeline	19
7	Discussion	20
7.1	Garbage collection	20
7.2	Dynamic rule sets	20
7.3	Equations, confluence, and FVP	20
7.4	Comparison to direct compilation	21
8	Conclusion	21

1 Introduction

Operational semantics for languages built on term rewriting typically consists of a finite signature, a finite set of equations, and a finite inductive specification of a reduction relation. The *interpreter* that implements such a semantics has two well-understood components:

1. a *matcher* that finds all redex occurrences in the subject term — the positions in the term where a left-hand side of some rewrite rule matches; and
2. a *rewriter* that fires a chosen redex by substituting the instance of the corresponding right-hand side and announces the positions in the resulting term where matching should resume.

Both components are eminently parallelisable: distinct positions in a term can be examined independently, and distinct redex sites in a non-overlapping rule set can be fired independently.

Two contributions of the last decade have sharpened the matcher side of this picture. First, the *set automaton* of Erkens and Groote [1] solves the subterm pattern-matching problem for a finite pattern set against a finite signature in a way that visits each function symbol of the subject term at most once and exposes the algorithm’s parallel structure intrinsically: at each state, the automaton emits a set of independent (state, position)-pairs that may be processed in any order, or simultaneously, by distinct workers. Second, the rewriter side has been put on a categorical footing by *graph-structured lambda theories* (GSLTs), which view an operational semantics as a category with finite limits equipped with a distinguished object of processes and a distinguished subobject for the reduction relation, presented finitely by generators, equations and inductive clauses.

This paper joins these two threads through Rholang, a concurrent process-calculus language [2] whose primitives are asynchronous send, persistent name-restricted receive, and reflective name/process quoting. Concretely, we give an algorithm

$$\text{compile} : \text{GSLTPresentation} \longrightarrow \text{RholangProgram}$$

with the following properties.

Totality.

`compile` is defined on every finitely presented GSLT, terminates, and runs in time polynomial in the size of the generated set automaton.

Correctness.

The Rholang program produced by `compile` implements the small-step semantics of the input presentation: for every input term encoded as a Rholang process, the program announces on a public channel exactly the set of redex (rule, position)-pairs given by the presentation, fires them in some sound order, and continues until normal form.

Parallelism preservation.

Every parallel opportunity exposed by the set automaton — independent successor positions, independent redexes, independent rewrites — is materialised as a parallel composition or as a free choice of message order in the Rholang output, without scheduling glue. The runtime is responsible for choosing an interleaving, and any choice is sound.

The technical core is the *reflection of automaton states as channels*. A set-automaton state is, by construction, a finite set of match obligations. We reflect this state, as a piece of syntactic data, to a Rholang name; the transitions out of the state become persistent receives on that name, the emission of (state, position)-successors becomes a parallel composition of sends to the successor states' names, and the announcement of a pattern match becomes a send on a public matches channel. The algorithm thus has a particularly compact closed form: every artefact of the compiler is one of a handful of *macros*, each templated by a state or a function symbol of the signature.

Outline. Section 2 fixes notation and recalls the three ingredients: GSLTs and their finite presentations (§2.1), the set automaton of Erkens–Groote (§2.2), and the fragment of Rholang we use (§2.3). Section 3 describes the encoding of subject terms as Rholang processes and of automaton states as Rholang names. Section 4 gives the compilation algorithm proper, organised as nine code-emission rules. Section 5 states and proves the parallelism-preservation theorem. Section 6 works through a small, complete example. Section 7 discusses garbage collection, dynamic rule sets, and other practical concerns. Section 8 concludes.

2 Preliminaries

2.1 Graph-structured lambda theories

A graph-structured lambda theory is a categorical packaging of an operational semantics: the category is the host of term constructors, the distinguished object is the universe of processes, and the distinguished subobject is the reduction relation.

Definition 2.1 (Graph-structured lambda theory). A *graph-structured lambda theory* (GSLT) is a tuple $\mathcal{G} = (\mathcal{C}, P, R)$ where

- $\mathcal{C}$ is a category with finite limits;
- $\mathcal{C}$ is closed with respect to the cartesian product (so internal homs $A \Rightarrow (-)$ exist as right adjoints);
- $P \in \mathcal{C}$ is a distinguished object, the *object of processes*;
- $R \hookrightarrow P \times P$ is a distinguished subobject, the *reduction relation*.

For implementation purposes we restrict to GSLTs given finitely.

Definition 2.2 (Finite presentation). A *finite presentation* of a GSLT consists of

1. a finite set $\mathcal{S}$ of *base sorts* (in the sense of a multisorted Lawvere theory [5]); we write $\mathcal{T}(\mathcal{S})$ for the closure of $\mathcal{S}$ under finite products $\times$ and exponentials $\Rightarrow$, and call its elements *types*;
2. a finite ranked alphabet $\mathbb{F} = \mathbb{F}_0 \cup \mathbb{F}_1 \cup \dots \cup \mathbb{F}_n$ of *function symbols* (term constructors), each with a fixed arity, a fixed product-shaped input/output profile $f : A_1 \times \dots \times A_{\#f} \rightarrow B$ with $A_1, \dots, A_{\#f}, B \in \mathcal{T}(\mathcal{S})$ (in particular each A_i may itself be a product or an exponential — for example $\text{lam} : (P \Rightarrow P) \rightarrow P$ for the λ -abstraction constructor, with input type the exponential $P \Rightarrow P$), and a finite set λ_f of *well-formedness equations* $g_i(\vec{x}) = h_i(\vec{x})$ between $\mathbb{F}$ -terms over the inputs $\vec{x} = (x_1, \dots, x_{\#f})$, which the inputs must satisfy modulo E for $f(\vec{x})$ to be well-formed;¹;
3. a finite set E of equations between $\mathbb{F}$ -terms, together with a reduction order $\succ$ on terms that orients E into a convergent (confluent and terminating) rewrite system $E^\downarrow$ via Knuth–Bendix completion, giving canonical representatives of equivalence classes; we further require that $E^\downarrow$ has the *finite variant property* (FVP) of Escobar–Meseguer–Sasse [3]: for every $\mathbb{F}$ -term t the set

$$\text{Var}_E(t) := \{(t\sigma) \downarrow_{E^\downarrow} : \sigma \text{ a substitution}\}$$

is finite up to renaming of metavariables. FVP makes E -matching — “does $P\sigma =_E T$ for some σ ?” — decidable via folding variant narrowing [4], and is enjoyed by all standard structural theories (AC, ACI, abelian groups, exclusive-or);²

4. a finite set $\mathcal{R}_{\text{rule}}$ of *rewrite rules* $L_j \rightsquigarrow R_j$, where L_j, R_j are morphisms in the syntactic Lawvere theory of $\mathbb{F}$ with a common typed domain $A_1 \times \dots \times A_m$ (the *metavariable inputs* of the rule) and codomain P , and a finite set $\mathcal{R}_{\text{ctx}}$ of *contextual rewrites* of the shape

$$L_1 \rightsquigarrow R_1, \dots, L_k \rightsquigarrow R_k \Rightarrow K[L_1, \dots, L_k] \rightsquigarrow K'[R_1, \dots, R_k],$$

with K, K' k -hole $\mathbb{F}$ -morphisms and each L_i, R_i an $\mathbb{F}$ -morphism in the same sense.

We display rules in the standard tree form, writing repeated metavariable tokens for the same input. A repeated occurrence of a metavariable $\omega : A$ on the LHS denotes composition with the diagonal $\Delta_A : A \rightarrow A \times A$ (or, more generally, an iterated Δ for several occurrences); the LHS

¹Definition 2.1 asks for $\mathcal{C}$ to have all finite limits, so in principle a function-symbol profile could be any finite limit $\lim D_f \rightarrow B$, not merely a product. By the standard product–equaliser factorisation of finite limits, every such profile is equivalent data to a product profile $f^\flat : A_1 \times \dots \times A_n \rightarrow B$ together with the finite set of equations λ_f defining the equaliser. We adopt that factorised form for compilation; no expressive power is lost. When $\lambda_f = \emptyset$ the profile is a plain product, as in a standard ranked alphabet.

²Without FVP, E -matching is in general undecidable even when $E^\downarrow$ is convergent, so KB alone is insufficient for the compiler to decide whether a subject term has an E -equivalent matching a rule’s left-hand side. Alternative sufficient conditions (left-linearity plus termination of basic narrowing; shallow E ; right-ground E) also work, but FVP is the cleanest single hypothesis.

as a whole is the categorically linear morphism that, after Δ -expansion, uses each metavariable input once. This unifies the two traditions of “linearity”: *categorical linearity* (the LHS is a well-formed morphism in the syntactic category, with Δ available because of finite products) is always assumed, and *syntactic linearity* (no metavariable token repeats) is the special case of categorical linearity in which no Δ -composition occurs;

5. a *reduction relation* $R \hookrightarrow P \times P$, specified as the least subobject closed under the following clauses, read pointwise on ground terms:

(Rule)

for every $(L_j \rightsquigarrow R_j) \in \mathcal{R}_{\text{rule}}$ and every ground substitution σ , $(L_j^\sigma, R_j^\sigma) \in R$;

(Context)

for every contextual rewrite

$$L_1 \rightsquigarrow R_1, \dots, L_k \rightsquigarrow R_k \Rightarrow K[L_1, \dots, L_k] \rightsquigarrow K'[R_1, \dots, R_k]$$

in $\mathcal{R}_{\text{ctx}}$ and every ground substitution σ , if $(L_i^\sigma, R_i^\sigma) \in R$ for all $1 \leq i \leq k$, then

$$(K^\sigma[L_1^\sigma, \dots, L_k^\sigma], K'^\sigma[R_1^\sigma, \dots, R_k^\sigma]) \in R.$$

We write $\mathcal{R} := \mathcal{R}_{\text{rule}} \cup \mathcal{R}_{\text{ctx}}$ for the full rule set of the presentation.

The presentation generates a GSLT freely by taking $\mathcal{C}$ to be the syntactic category of the simply-typed λ -theory with constants $\mathbb{F}$, modulo E and $\beta\eta$; P to be a base sort, and R to be the least subobject closed under (Rule) and (Context).

Binders. A function symbol $f \in \mathbb{F}_n$ may carry a set $\beta_f \subseteq \{1, \dots, n\}$ of *binding positions*. Under the internal hom of Definition 2.1, the i -th argument of f for $i \in \beta_f$ has type $P \Rightarrow P$. Operationally, the subterm at position $p.i$ of $f(t_1, \dots, t_n)$ for $i \in \beta_f$ is the opened body $t_i(z_{p.i})$ for a fresh free name $z_{p.i}$; the position-domain, sub-pattern, and head-symbol functions extend by the obvious recursion through opened bodies. Free names introduced by binders behave as leaves in the term tree, analogous to variables. The set-automaton machinery and the compiler we describe are agnostic to binders: positions and head symbols are the only operationally relevant features, and binders contribute only to the way ground substitutions interact with rule firings (cf. §4.6).

Patterns, left-hand sides, and sharing constraints. A *raw pattern* is a tree in $\mathbb{T}(\mathbb{F}, \mathcal{V})$ over a countable set $\mathcal{V}$ of metavariables, with metavariables permitted to repeat. The *linearisation* of a raw pattern ℓ is the pair $(\ell^\flat, \kappa_\ell)$ where $\ell^\flat$ replaces every occurrence of every metavariable by a distinct fresh variable, and κ_ℓ is the set of position-pairs (p_1, p_2) such that the p_1 -leaf and p_2 -leaf of ℓ carry the same original metavariable. We call κ_ℓ the *sharing constraint* of ℓ ; it is empty iff ℓ is syntactically linear. The set of left-hand sides of a presentation is

$$\mathcal{L} = \{(\ell^\flat, \kappa_\ell) : \ell \in \text{LHSes}(\mathcal{R}_{\text{rule}}) \cup \text{LHSes}(\mathcal{R}_{\text{ctx}})\},$$

and $\mathcal{L}$ is the input to the set-automaton construction below. We typically drop the $\flat$ superscript and write ℓ when the distinction is clear from context.

2.2 The set automaton

The set automaton of Erkens and Groote [1] solves the *subterm pattern-matching problem*: given a finite pattern set $\mathcal{L}$, locate every (ℓ, p) -pair such that pattern $\ell \in \mathcal{L}$ matches the subject term at position p . We recall the construction at the level of detail needed for compilation; for proofs and intuitions we refer to the original paper.

Match obligations and goals. A *match obligation* on $\mathcal{L}$ is a finite, non-empty subset of $\text{sub}(\mathcal{L}) \times \mathbb{P}$, where $\text{sub}(\mathcal{L})$ is the set of sub-patterns of patterns in $\mathcal{L}$ excluding the metavariable ω , and $\mathbb{P}$ is the set of positions (finite sequences of positive integers, with ϵ for the root and $p.q$ for concatenation). A *match announcement* is a pair $(\ell, p) \in \mathcal{L} \times \mathbb{P}$. A *match goal* is an arrow $mo \rightarrow ma$ from a match obligation to a match announcement, written

$$\ell_1 @ p_1, \dots, \ell_n @ p_n \rightarrow \ell @ p,$$

read: *in order to announce that ℓ matches at p , we must observe sub-pattern ℓ_i at p_i for each $1 \leq i \leq n$.*

The tuple. A *set automaton* for $\mathcal{L}$ is a tuple $M = (\mathcal{S}, s_0, L, \delta, \eta)$ where:

- $\mathcal{S}$ is a finite set of states, each state itself a finite non-empty set of match goals;
- $s_0 = \{\ell @ \epsilon \rightarrow \ell @ \epsilon \mid \ell \in \mathcal{L}\}$ is the initial state;
- $L : \mathcal{S} \rightarrow \mathbb{P}$ is the *state-label function*, assigning to every state a position in some match obligation (the position to inspect next);
- $\delta : \mathcal{S} \times \mathbb{F} \rightarrow \mathcal{P}(\mathcal{S} \times \mathbb{P})$ is the *transition function*;
- $\eta : \mathcal{S} \times \mathbb{F} \rightarrow \mathcal{P}(\mathcal{L} \times \mathbb{P} \times \mathcal{K})$ is the *output function*, emitting triples (ℓ, p, κ) where $\kappa \in \mathcal{K}$ is the sharing constraint inherited from ℓ , instantiated at the announcement position p .

The empty set is the *final state*: no transitions, no output.

Operation. The automaton is run with a *position pointer* p ranging over positions in the subject term t . A configuration (s, p) at state s and pointer p proceeds by reading the head symbol $f := \text{hd}(t[p.L(s)])$ at position $p.L(s)$ in t ; then:

- for every $(\ell, p', \kappa) \in \eta(s, f)$, announce that pattern ℓ matches at position $p.p'$, conditioned on the sharing constraint κ holding at $p.p'$ in t ; if κ is empty the announcement is unconditional;
- for every $(s', p'') \in \delta(s, f)$, recursively process configuration $(s', p.p'')$.

The order in which the successor configurations $\{(s', p.p'') : (s', p.p'') \in \delta(s, f)\}$ are processed is unspecified. Sequential implementations use a stack or queue, giving DFS or BFS; the crucial observation [1] is that any order is sound and visits each function symbol of t at most once.

Construction. The states, label function, and transition function are jointly defined as a least fixed point. Given a state s and a function symbol $f \in \mathbb{F}$, the *f-derivative* of s is the next-step state obtained by updating all match obligations in s that mention $L(s)$ under the observation of f at $L(s)$:

$$\text{deriv}(s, f) = \text{unchanged} \cup \text{reduced} \cup \text{fresh},$$

where (using the obvious component-wise reduction operator on obligations)

$$\text{unchanged} = \{mo \rightarrow ma \in s \mid L(s) \notin \text{pos}(mo)\},$$

$$\text{reduced} = \{\text{reduce}(mo, f, L(s)) \rightarrow ma \mid mo \rightarrow ma \in s, \exists (\ell @ L(s)) \in mo : \text{hd}(\ell) = f, \text{reduce}(mo, f, L(s)) \neq \emptyset\},$$

$$\text{fresh} = \{\ell @ L(s).i \rightarrow \ell @ L(s).i \mid \ell \in \mathcal{L}, 1 \leq i \leq \#f\}.$$

The derivative is then partitioned into dependency-equivalence classes (two obligations are dependent iff their position-sets intersect, and the equivalence is the transitive closure of dependency); each class becomes a new state after *lifting*, i.e. replacing each position by its image under the greatest-common-prefix offset. Lifting keeps the position pointers bounded and the state set finite. The construction is deterministic up to the (immaterial) choice of $L(s)$ when several root obligations are available.

Sharing constraints. The construction above is unchanged from the syntactically linear case of Erkens–Groote [1]: it runs on $\mathcal{L}$ viewed as a set of linearised raw patterns ℓ^b , ignoring the sharing constraints κ_ℓ . The constraints reappear at output time: when state s has a root goal whose announcement fires under symbol f at position p , the triple emitted is (ℓ, p', κ) where κ is κ_ℓ with every position q in its pairs replaced by $p.q$ (i.e., reanchored to the announcement position in the subject term). The set automaton is therefore agnostic to Δ : it detects syntactic shape and defers structural-equality checking to the fire stage (§4.6).

Example: the rho COMM rule. The rho-calculus communication rule

$$\text{for}(y \leftarrow x)\{B\} \mid x!(Q) \rightsquigarrow B[@Q/y]$$

is the prototypical non-syntactically-linear LHS: as a morphism in the syntactic Lawvere theory it has shape $N \times (N \Rightarrow P) \times P \rightarrow P$ with one channel-input x , one body-input B (binding y), and one payload Q , but its tree display has two leaves labelled x (composition with Δ_N). The linearisation introduces fresh metavariables $x_1, x_2 : N$ at positions 1.1 (subject of the receive) and 2.1 (subject of the send), giving sharing constraint

$$\kappa_{\text{COMM}} = \{(1.1, 2.1)\}.$$

The set automaton is built on ℓ_{COMM}^b in the usual way; at announcement of ℓ_{COMM}^b at some position p in the subject, the emitted triple is $(\text{COMM}, p, \{(p.1.1, p.2.1)\})$, and the fire contract (§4.6) verifies $p.1.1 \equiv p.2.1$ in the subject before firing.

2.3 Rholang

We use a small fragment of Rholang based on the rho calculus of Meredith and Radestock [2]. Processes P and names x are mutually defined by the grammar

$$P ::= \mathbf{0} \mid x!(P) \mid \text{for}(y \leftarrow x)\{P\} \mid \text{for}(y \Leftarrow x)\{P\} \mid P \mid P \mid *x \mid \text{new } x \text{ in } \{P\}, \quad x ::= @P,$$

with the following intended readings:

- $\mathbf{0}$: the inert process;
- $x!(P)$: asynchronous send of process P on channel x ;
- $\text{for}(y \leftarrow x)\{P\}$: *linear* receive on channel x , binding the incoming name to y in P and then consuming the receiver;
- $\text{for}(y \Leftarrow x)\{P\}$: *persistent* receive (*contract*), which fires on every send to x , binding y in P , and remains available;
- $P \mid Q$: parallel composition;

- $*x$: dereference of name x , evaluating to the process it names;
- $\text{new } x \text{ in } \{P\}$: name restriction, introducing a fresh name x in P ;
- $@P$: the *reflection* of process P to a name.

Structural congruence $\equiv$ contains α -equivalence, associativity and commutativity of $|$ with $\mathbf{0}$ as unit, and the reflection law $*@P \equiv P$. Names are quotients of processes under $\equiv$. The principal reduction is

$$\text{for}(y \leftarrow x) \{P\} \mid x'!(Q) \rightsquigarrow P[@Q/y] \quad \text{whenever} \quad *x \equiv *x',$$

with the persistent variant leaving a fresh copy of the receive in place.

We use Rholang's standard surface syntax in displayed code, writing $\mathbf{x}!(P)$ for $x!(P)$, $\text{for}(\mathbf{y} \leftarrow \mathbf{x}) \{P\}$ for the linear receive and $\text{for}(\mathbf{y} \leq \mathbf{x}) \{P\}$ for the persistent variant, and $\text{new } \mathbf{x} \text{ in } \{P\}$ for $\text{new } x \text{ in } \{P\}$.

Match. Surface Rholang adds a `match` construct on quoted-process patterns. We use `match` only to deconstruct the surface symbol and the arguments of an encoded subject term (§3); it is unnecessary for the set-automaton dispatch, which is realised entirely by channel names.

Reflection bracket. Throughout the paper $\lceil \cdot \rceil$ denotes a fixed section of $@$ chosen so that distinguishable mathematical objects map to distinct names: $\lceil a \rceil =_{\mathcal{N}} \lceil b \rceil$ iff a and b are equal as mathematical objects. We use the bracket on automaton states, term positions, and signature symbols, generating canonical and distinct channel names from each.

3 Encoding terms and states

The compiler emits a fixed set of definitions independent of the input term, plus an encoding of each subject term as a Rholang process. We describe the encodings here; the compilation algorithm proper is in §4.

3.1 Subject terms

Let $t \in \mathbb{T}(\mathbb{F})$ be a closed subject term. We encode t as a Rholang process on a *term channel* τ that, when read, yields a constructor tag together with a tuple of names for the immediate subterms. Concretely, for each function symbol $f \in \mathbb{F}_n$ we emit a persistent receive that responds to a request on τ with the head symbol f and a tuple of names $(\tau_1, \dots, \tau_n)$ whose targets are the recursively encoded subterms.

Construction 3.1 (Term encoding). The encoding $\llbracket t \rrbracket_\tau$ of a term t on a channel name τ is defined inductively:

- For $a \in \mathbb{F}_0$ (a constant):

$$\llbracket a \rrbracket_\tau = \text{for}(r \leftarrow \tau) \{r!(\lceil a \rceil)\}.$$

- For $f \in \mathbb{F}_n$ with $n \geq 1$ and subterms $t_1, \dots, t_n$:

$$\llbracket f(t_1, \dots, t_n) \rrbracket_\tau = \text{new } \tau_1, \dots, \tau_n \text{ in } \{ \llbracket t_1 \rrbracket_{\tau_1} \mid \dots \mid \llbracket t_n \rrbracket_{\tau_n} \mid \text{for}(r \leftarrow \tau) \{r!(\lceil f \rceil, \tau_1, \dots, \tau_n)\} \}.$$

A query on τ thus returns the *shape* of t at the root — its head symbol plus channel names addressing its arguments — and leaves sub-shapes available behind those argument channels. This makes the encoding lazy in a useful sense: the automaton consults only the positions it actually needs to inspect.

Binders. A function symbol $f \in \mathbb{F}_n$ with binding positions $\beta_f \subseteq \{1, \dots, n\}$ encodes binding arguments as $(P \Rightarrow P)$ -valued, i.e. as Rholang names that, when sent a fresh *bound-name token*, respond with the encoding of the opened body under that token. Concretely, position $i \in \beta_f$ of f is encoded on a channel τ_i as a persistent receive

$$\text{for } ((z, r) \leftarrow \tau_i) \{ r!([\![t_i(z)]\!]_r) \},$$

which takes a fresh free name z and a response channel r , and emits the encoded opened body. The automaton invokes this protocol only at positions in β_f that it actually inspects.

3.2 Automaton states as channels

For each state $s \in \mathcal{S}$ of the set automaton built from $\mathcal{L}$, the compiler reserves a single *state channel* $\sigma_s := \ulcorner s \urcorner$. The state channel carries messages of the form (τ, p) , where τ is the channel encoding the subject term and p is a position pointer into t .

A persistent receive on σ_s implements the per-state behaviour: the position $L(s)$ is added to p to obtain the position to inspect; the term is queried at that position, yielding a head symbol f and the argument channels; $\eta(s, f)$ is announced on a public *matches* channel; and each $(s', p'') \in \delta(s, f)$ is sent forward as $(\tau, p.p'')$ on $\sigma_{s'}$. Section 4 renders this behaviour as explicit Rholang code.

3.3 Position pointers

A position pointer $p \in \mathbb{P}$ is encoded as a Rholang list of positive integers, or, equivalently, as a name $\ulcorner p \urcorner$. Concatenation $p.q$ is a Rholang list operation. Lookup at a position is realised by a sequence of *traversal* sends down the argument channels: from a term channel τ and a pointer $p = i_1.i_2 \dots i_k$, the sequence

$$\tau!(r_0), \text{ for } ((f_0, \vec{\tau}_0) \leftarrow r_0) \{ \vec{\tau}_0[i_1]!(r_1), \text{ for } ((f_1, \vec{\tau}_1) \leftarrow r_1) \dots \}$$

descends the term tree along p . This traversal is generic and is emitted once by the compiler as a contract (the *traversal contract*, see §4.3).

4 The compilation algorithm

We are now ready to state the algorithm. The input is a finite GSLT presentation Π ; the output is a Rholang program $\text{compile}(\Pi)$. The compiler is presented as nine emission rules, each of which produces a piece of Rholang code parameterised over data extracted from Π and the set automaton $M = (\mathcal{S}, s_0, L, \delta, \eta)$ built from $\mathcal{L}(\Pi)$.

Algorithm 4.1 ($\text{compile}(\Pi)$).

Input: a finite GSLT presentation $\Pi = (\mathcal{S}, \mathbb{F}, E, \succ, \mathcal{R}_{\text{rule}}, \mathcal{R}_{\text{ctx}})$.

Output: a closed Rholang program.

Steps.

1. *Normalise equations.* Use Knuth–Bendix on $(E, \succ)$ to obtain a confluent, terminating rewrite system $E^\downarrow$ whose normal forms are the canonical representatives modulo E . The remaining steps assume that all left-hand sides in $\mathcal{R}_{\text{rule}} \cup \mathcal{R}_{\text{ctx}}$ are in $E^\downarrow$ -normal form.
2. *Build $\mathcal{L}$.* Collect the left-hand sides of $\mathcal{R}_{\text{rule}}$ and the per-premise left-hand sides of $\mathcal{R}_{\text{ctx}}$.
3. *Build the set automaton M .* Compute the states, L , δ , and η by the construction of §2.2.

4. *Emit the runtime preamble* (§4.1).
5. *Emit the term constructors* (§4.2).
6. *Emit the traversal contract* (§4.3).
7. *Emit the state contracts* (§4.4).
8. *Emit the equational normaliser* (§4.5).
9. *Emit the rewriter* (§4.6).
10. *Emit the driver* (§4.7).

The order of emission is not significant; emitted contracts are persistent and unordered.

We describe each emission rule in turn. All emitted code lives inside a single top-level **new** block that introduces the channel names used throughout. We elide that wrapper in displayed code.

4.1 Preamble

The preamble declares the public channels:

- **matches** — a public channel on which the interpreter announces every (L, p) -pair such that L matches the current subject term at position p ;
- **fire** — a public channel on which a scheduler may instruct the interpreter to fire a specific announced redex;
- **normalForm** — a public channel that receives the final term when no more redexes are available.

```
new matches, fire, normalForm,
  /* state channels: */ ${state-names},
  /* helpers: */ traverse, substitute, normalise, driver in {
  // ... remainder of the program ...
}
```

The placeholder $\${state-names}$ expands to the list $\sigma_{s_0}, \sigma_{s_1}, \dots, \sigma_{s_{|S|-1}}$.

4.2 Term constructors

For each $f \in \mathbb{F}_n$ the compiler emits a *constructor contract* that builds a term-channel responder for f :

```
for (@(t1, ..., tn), r <= mk_${f}) {
  new tau in {
    for (reply <= tau) { reply!($f-tag, t1, ..., tn) } |
    r!(tau)
  }
}
```

The placeholder $\${f-tag}$ is the canonical name $\ulcorner f \urcorner$. A constant $a \in \mathbb{F}_0$ emits the obvious zero-argument variant.

For each $f \in \mathbb{F}_n$ with binding positions β_f , the $i \in \beta_f$ slot is a name τ_i that satisfies the bound-body protocol described in §3.1; the constructor contract above accepts such a name without further change.

Well-formedness equations. When λ_f is non-empty, the constructor contract guards f -formation on the input equations: it invokes `checkConstraint` (§4.6) on the inputs $(t_1, \dots, t_n)$ against λ_f before publishing the new term channel, and emits a typing failure on r if any equation fails. The check is structurally identical to the sharing-constraint check of §4.6: each equation $g_i(\vec{x}) = h_i(\vec{x})$ is evaluated to two term channels and compared by Rholang structural congruence. When $\lambda_f = \emptyset$ the guard is elided and the contract reduces to the form above.

4.3 Traversal contract

A single *traversal contract* consumes a term channel and a position list and replies with the term channel at that position:

```
for (@(tau, p), reply <= traverse) {
  match p {
    Nil      => { reply!(tau) }
    [i, ...q] => {
      new r in {
        tau!(r) |
        for ((_head, @args) <- r) { traverse!((args.nth(i), q), reply) }
      }
    }
  }
}
```

Note. The pattern $(_head, @args)$ extracts the head tag and the tuple of argument channels; `args.nth(i)` selects the i -th of them. The traversal is asynchronous: every step requests the next argument channel via a fresh response name r and resumes only when the term channel replies.

4.4 State contracts

The set-automaton dispatch is the heart of the compiler. For each non-final state $s \in \mathcal{S}$ we emit a single persistent contract on the state channel σ_s :

```
for (@(tau, p) <= ${state-name(s)}) {
  new r, posChan in {
    // 1. Compute the inspection position p . L(s):
    posChan!(p ++ ${pos-literal(L(s))}) |
    for (@inspectPos <- posChan) {
      // 2. Ask traverse for the term channel at inspectPos:
      traverse!((tau, inspectPos), r) |
      for (subTau <- r) {
        new rr in {
          subTau!(rr) |
          for ((@head, @args) <- rr) {
            // 3. Dispatch on the head symbol:
            match head {
              ${case-for-each-f-in-F(s)}
              _ => Nil
            }
          }
        }
      }
    }
  }
}
```

}

The placeholder $\$\{\text{case-for-each-f-in-F}(s)\}$ expands to one `match` arm per function symbol f for which $\delta(s, f) \cup \eta(s, f)$ is non-empty. The arm has the form

```

 $\{\text{f-tag}\} \Rightarrow \{
  // (a) emit pattern-match announcements eta(s, f):
  \{\text{emit-eta}(s, f)\} \mid
  // (b) emit transitions delta(s, f), in parallel:
  \{\text{emit-delta}(s, f)\}
\}$ 
```

where

$$\text{emit-eta}(s, f) = \big|_{(\ell, p', \kappa) \in \eta(s, f)} \text{matches}!(\ulcorner \ell \urcorner, p \mathrel{++} p', \ulcorner \kappa \langle p.p' \rangle \urcorner)$$

and

$$\text{emit-delta}(s, f) = \big|_{(s', p'') \in \delta(s, f)} \sigma_{s'}!(\tau, p \mathrel{++} p'').$$

The notation $\kappa \langle q \rangle$ denotes κ with every position-pair (q_1, q_2) replaced by $(q.q_1, q.q_2)$ (the reanchoring at the announcement position). When κ is empty (i.e., the LHS is syntactically linear), $\ulcorner \kappa \langle \cdot \rangle \urcorner$ is a canonical name for the empty constraint, and the fire contract (§4.6) bypasses the equality check. The vertical bar denotes Rholang parallel composition: distinct announcements and distinct successor configurations are emitted in parallel and are scheduled by the Rholang runtime.

The final state (the empty set of match goals) emits no contract: a configuration that arrives at the final state simply terminates that branch of the search.

What this code is doing. Each state contract reads a configuration (τ, p) and:

1. computes the inspection position $p.L(s)$;
2. asks the traversal contract for the term channel at that position;
3. asks that channel for its head symbol f and argument channels;
4. announces all matches $\eta(s, f)$ in parallel;
5. dispatches all successor configurations $\delta(s, f)$ in parallel.

There is no shared state and no scheduling barrier. The arrows of $\delta(s, f)$ correspond to distinct messages on distinct or identical channels, all available simultaneously to any Rholang reducer.

4.5 Equational normaliser

The equations E of the presentation, after Knuth–Bendix completion, form a convergent rewrite system $E^\downarrow$ that gives a canonical representative for each equivalence class. Because Definition 2.2 requires $E^\downarrow$ to satisfy the finite variant property, a subject term and its E -canonical form admit the *same* set of pattern matches against any left-hand side of $\mathcal{R}$ (in the sense of E -matching): normalising before dispatch is sound and complete. The compiler treats $E^\downarrow$ as a sub-presentation whose set automaton $M^\downarrow$ matches the left-hand sides of $E^\downarrow$ in the subject term, and whose rewriter (§4.6) fires them eagerly. We emit a *normaliser* contract that, given a term channel, repeatedly fires $E^\downarrow$ -redexes until none remain and then signals completion:

```

for (tau, done <= normalise) {
  // Run the E-downward set automaton, fire its redexes innermost-first,
  // and signal "done" when matches is exhausted on E-downward patterns.
  // ...
}

```

The full body is structurally identical to the driver in §4.7; we omit it. Normalisation is interleaved with $\mathcal{R}$ -rewriting in the driver.

4.6 Rewriter

For each rule $L_j \rightsquigarrow R_j \in \mathcal{R}_{\text{rule}}$ the compiler emits a *fire contract* parameterised by the pattern identifier $\lceil L_j \rceil$. The contract takes the announcement triple (τ, p, κ) produced by the matcher; if κ is non-empty, the contract first verifies the structural equalities it specifies and aborts the firing if any fails. The Rholang skeleton is:

```

for (@(tau, p, kappa), done <= fire_{$\{L_j\}}) {
  new sigma, rTau, eqOk in {
    // 0. Verify sharing constraints kappa, if any:
    checkConstraint!((tau, kappa), eqOk) |
    for (@ok <- eqOk) {
      if (ok) {
        // 1. Extract the substitution sigma from tau at p against L_j:
        extractSubst!((tau, p, ${pattern-id(L_j)}), sigma) |
        for (@s <- sigma) {
          // 2. Build the right-hand side R_j under sigma on a fresh
          //    channel:
          buildRhs!((${rhs-id(R_j)}), @s, rTau) |
          // 3. Splice rTau into tau at p:
          for (newTau <- rTau) {
            splice!((tau, p, newTau), done)
          }
        }
      } else {
        done!(@"abort")
      }
    }
  }
}

```

The auxiliary contracts `checkConstraint`, `extractSubst`, `buildRhs`, and `splice` are generic and emitted once each.

Constraint check. The `checkConstraint` contract is the workhorse for every equality side-condition emitted by the compiler. It receives a term channel τ and a finite set of *equality items* $\xi = \{(e_1, e'_1), \dots, (e_m, e'_m)\}$ and replies `true` iff $e_i \equiv e'_i$ in τ for every i , where each e_i is either:

- a *position pair*, as used in sharing constraints κ and resolved by `traverse` into a sub-channel of τ ; or
- an $\mathbb{F}$ -term over the inputs of a constructor, as used in well-formedness equations λ_f and resolved by `buildRhs` into a freshly built term channel.

In both cases, each pair-check is realised by two parallel resolution steps returning channels c_i, c'_i and a single Rholang **match** that succeeds iff $c_i \equiv c'_i$ as quoted processes (a decidable test on rho-calculus structural congruence). The m pair-checks run in parallel; **checkConstraint** joins their booleans by conjunction. The constructor contracts of §4.2 invoke **checkConstraint** with $\xi = \lambda_f$ resolved on the inputs; the fire contracts above invoke it with $\xi = \kappa$ resolved on the subject channel.

Substitution and binders. The auxiliary **buildRhs** contract takes a substitution σ (a mapping from metavariables of L_j to term channels) and the static syntactic data of R_j , and produces a term channel for R_j^σ . For a binding occurrence in R_j — a metavariable ω that occurs under a binding position of some constructor — the corresponding channel from σ already carries a bound-name token in its protocol; **buildRhs** composes it with the surrounding constructor contracts straightforwardly.

Contextual rewrites. A contextual rewrite

$$L_1 \rightsquigarrow R_1, \dots, L_k \rightsquigarrow R_k \Rightarrow K[L_1, \dots, L_k] \rightsquigarrow K'[R_1, \dots, R_k]$$

is compiled in two stages. The premise patterns L_i are included in $\mathcal{L}$ and matched by the set automaton as in any other rule. The contextual conclusion fires only when announcements of all k premises arrive on **matches** in a configuration that places them at the k holes of K ; this synchronisation is implemented as a *join* on k messages on the **matches** channel, gated by a check that the positions are compatible with K . The check is itself a finite-state predicate on positions, emitted by the compiler from K .

4.7 Driver

The driver wires the components together. Given an initial subject term t_0 , the driver:

1. builds the term channel τ_0 for t_0 via the constructor contracts;
2. normalises τ_0 with the equational normaliser;
3. kicks off the set automaton by sending (τ_0, ϵ) on σ_{s_0} ;
4. collects announcements on **matches** into a redex pool;
5. upon receiving an instruction on **fire**, dispatches the appropriate fire contract;
6. after each firing, restarts the automaton from σ_{s_0} on the new term channel (or, with a more refined construction, from the partial-evaluation state determined by the rewritten position);
7. when **matches** reports empty after a full automaton run, sends the current term on **normalForm** and terminates.

The default scheduling policy is to forward every announcement back as an instruction to fire, in some order; alternative policies (outermost, innermost, etc.) are obtained by filtering or sorting the redex pool before forwarding.

5 Parallelism preservation

The defining feature of the set automaton is that it exposes parallelism in two places: *intra-state* (the elements of $\delta(s, f)$ and $\eta(s, f)$ are independent and may be processed in any order) and *inter-rule* (announced matches at non-overlapping positions are independent redexes). We show that both kinds of parallelism are preserved verbatim by Algorithm 4.1.

5.1 Concurrency model

Recall that the Rholang runtime is asynchronous and name-passing: a send $x!(P)$ may be paired with any matching receive on x , in any order, and parallel composition $P_1 \mid P_2$ enforces no ordering. The reduction relation of the rho calculus is closed under parallel composition: $P \rightsquigarrow P'$ implies $P \mid Q \rightsquigarrow P' \mid Q$. Two parallel sub-processes may reduce independently and concurrently, so long as their reductions do not compete for the same linear receive.

Definition 5.1 (Independent reductions). Two reductions $P_1 \rightsquigarrow P'_1$ and $P_2 \rightsquigarrow P'_2$ of sub-processes of P are *independent* if they involve disjoint sets of names and disjoint sub-processes.

5.2 Intra-state parallelism

Theorem 5.2 (Intra-state parallelism). *Let s be a state of the set automaton and f a function symbol such that $\delta(s, f) = \{(s'_1, p''_1), \dots, (s'_m, p''_m)\}$ and $\eta(s, f) = \{(\ell_1, q_1), \dots, (\ell_k, q_k)\}$. After dispatch on the head symbol f at a configuration (τ, p) , the corresponding Rholang sub-process is*

$$\left| \prod_{j=1}^k \text{matches}!((\ulcorner \ell_j \urcorner, p.q_j)) \right| \left| \prod_{i=1}^m \sigma_{s'_i}!((\tau, p.p''_i)) \right|.$$

The $k + m$ resulting message sends are pairwise independent in the sense of Definition 5.1; the Rholang runtime may schedule them in any order or concurrently, and the resulting global behaviour is identical to that of the set automaton run with any sequential schedule of the corresponding (s', p) -successors and (ℓ, p) -announcements.

Proof. The $k + m$ message sends share only the read-only names τ and p (the latter as a fresh local list, not a Rholang channel target); the target channels matches and $\sigma_{s'_1}, \dots, \sigma_{s'_m}$ are pairwise distinct or, in the case that two transitions go to the same state $s'_i = s'_j$, both receive messages on the same persistent contract — which by definition processes them in some order without interference. Soundness of any scheduling order is exactly the soundness theorem for the set automaton [1, Theorem 6.1]; that the Rholang runtime may schedule any order is immediate from the asynchronous semantics of $\mid$ and the persistence of the state contracts. $\square$

5.3 Inter-rule parallelism

Theorem 5.3 (Inter-rule parallelism). *Let Π be a finite GSLT presentation and let t be a subject term. Let*

$$A = \{(\ell, p) : \text{the set automaton announces } \ell \text{ at } p \text{ during the run on } t\}$$

be the set of announcements produced by the compiled Rholang program in its first automaton phase. The Rholang process announcing A is of the form

$$\left| \prod_{(\ell, p) \in A} \text{matches}!((\ulcorner \ell \urcorner, p)) \right| \mid (\text{automaton continuations}),$$

and the announcements are pairwise independent. Any subset of redexes fired by the driver in parallel (a parallel firing $\{(\ell_{i_1}, p_{i_1}), \dots, (\ell_{i_r}, p_{i_r})\}$ with pairwise non-overlapping positions) is realised by the Rholang runtime as the parallel firing of r independent fire-contract invocations, without inter-firing scheduling glue.

Proof. The set of announcements is the parallel composition of independent sends to `matches` by Theorem 5.2 composed transitively over the automaton run. The driver dispatches each fire instruction to a fresh `fire $\ell$` contract invocation, which allocates its own `rTau`, `sigma`, and other local channels via `new`, so no two fire contracts share linear channels. Non-overlapping positions imply non-overlapping subject-term sub-channels, which by Construction 3.1 are allocated under distinct `new`-restrictions. Hence the r firings touch disjoint sets of names and disjoint sub-processes, and proceed independently. $\square$

5.4 Why this matters

The combined effect of Theorems 5.2 and 5.3 is that no part of the compiled interpreter contains a centralised scheduler. The matcher emits successor configurations as parallel sends; the rewriter emits parallel firings as parallel sub-processes; the driver merely forwards. Concretely, the only points at which the interpreter is sequential are the points at which the set automaton is inherently sequential — namely, the inspection of the head symbol of a single sub-term at a single position. Every other step is parallel by construction, and Rholang’s runtime is responsible for choosing any interleaving.

Corollary 5.4 (No-glue corollary). *The compiled Rholang program contains no parallel-scheduling code: the emitted runtime preamble, traversal contract, state contracts, fire contracts, and driver are all sequential within their bodies, and all inter-component parallelism is exposed solely through Rholang’s $|$ -composition. In particular, the size of the compiled program is $O(|\mathcal{S}| \cdot |\mathbb{F}| + |\mathcal{R}| + |\mathbb{F}|)$, with no multiplicative factor for scheduling.*

6 A worked example

We illustrate the algorithm on a tiny presentation: the associativity patterns of Erkens–Groote [1, §3], treated as a rewrite system that re-associates a binary operator to the right.

6.1 The presentation

Sorts: $\mathcal{S} = \{\iota\}$. Signature: $\mathbb{F}_0 = \{a\}$, $\mathbb{F}_2 = \{f\}$. Equations: none beyond the trivial. Rules:

$$\mathcal{R}_{\text{rule}} = \{ L_1 \rightsquigarrow R_1 \} \quad \text{where} \quad L_1 = f(f(\omega_1, \omega_2), \omega_3), \quad R_1 = f(\omega_1, f(\omega_2, \omega_3)).$$

Contextual rewrites: none. The left-hand-side set is $\mathcal{L} = \{L_1\}$. (The Erkens–Groote example uses a second pattern $L_2 = f(\omega_1, f(\omega_2, \omega_3))$ to demonstrate set automata; we drop it here so the worked example fits on one page.)

6.2 The set automaton

Applying the construction of §2.2 to $\mathcal{L} = \{L_1\}$ yields:

- Initial state $s_0 = \{L_1 @ \epsilon \rightarrow L_1 @ \epsilon\}$ with $L(s_0) = \epsilon$;

- f -derivative of s_0 contains the new goal $f(\omega_1, \omega_2)@1 \rightarrow L_1@e$ and fresh goals $L_1@1 \rightarrow L_1@1$, $L_1@2 \rightarrow L_1@2$; partitioning and lifting yield successors at position pointer 1 and 2;
- further derivatives yield a finite set of states with position labels in $\{e, 1, 2\}$ and announcements $\eta(s, f) = \{(L_1, e)\}$ at the (unique) state s whose obligations require both $\text{hd}@e = f$ and $\text{hd}@1 = f$.

We abbreviate the resulting four-state automaton as $\mathcal{S} = \{s_0, s_1, s_2, s_3\}$.

6.3 The compiled Rholang program

We give the key fragments. The preamble:

```
new matches, fire, normalForm,
    s0, s1, s2, s3,           // state channels
    mk_a, mk_f,               // constructors
    traverse, fire_L1,
    extractSubst, buildRhs, splice,
    driver in {
    // ... contracts below ...
    driver!()
}
```

The constructors:

```
for (r <= mk_a) {
  new tau in {
    for (reply <= tau) { reply!("@a",)) } |
    r!(tau)
  }
} |
for (@(t1, t2), r <= mk_f) {
  new tau in {
    for (reply <= tau) { reply!("@f", t1, t2)) } |
    r!(tau)
  }
}
```

The state contract for s_0 (label e , f -transition to s_1 at position 1 and s_2 at position 2, with the announcement (L_1, e) deferred to a deeper state):

```
for (@(tau, p) <= s0) {
  new r in {
    traverse!((tau, p), r) |           // L(s_0) = epsilon, so p ++ [] = p
    for (subTau <- r) {
      new rr in {
        subTau!(rr) |
        for ((@head, @args) <- rr) {
          match head {
            @"f" => {
              s1!((tau, p ++ [1])) | // descend to position p.1
              s2!((tau, p ++ [2]))   // descend to position p.2
            }
            - => Nil
          }
        }
      }
    }
  }
}
```

```

    }
  }
}

```

The state contract for the state s at which the announcement fires:

```

for (@(tau, p) <= s1) {
  new r in {
    traverse!((tau, p), r) |           // L(s_1) = epsilon
    for (subTau <- r) {
      new rr in {
        subTau!(rr) |
        for ((@head, @args) <- rr) {
          match head {
            @"f" => {
              // announce: L_1 matches at p.epsilon (= p)
              matches!("@L_1", p)) |
              // and continue the inner search:
              s1!((tau, p ++ [1])) |
              s2!((tau, p ++ [2]))
            }
            - => Nil
          }
        }
      }
    }
  }
}

```

(Note. The redex announcement and the inner search are emitted in parallel. The Rholang runtime is free to deliver them in any order; both will eventually run.)

The fire contract for L_1 , sketched:

```

for (@(tau, p), done <= fire_L1) {
  new sigma, rTau in {
    extractSubst!((tau, p, @"L_1"), sigma) |
    for (@s <- sigma) {
      // s = { omega_1 /-> t11, omega_2 /-> t12, omega_3 /-> t2 } where
      // t = f(f(t11, t12), t2)
      mk_f((s.omega_2, s.omega_3), @"inner") |
      for (innerTau <- @"inner") {
        mk_f((s.omega_1, innerTau), rTau) |
        for (newTau <- rTau) {
          splice!((tau, p, newTau), done)
        }
      }
    }
  }
}

```

(In a production compiler the literal name `@inner` would be a fresh `new`-bound name; we use the literal here for readability.)

The driver simply forwards every announcement to a fire instruction and restarts the automaton:

```

for (_ <= driver) {
  // ... build initial tau_0 from input ...
}

```

```

s0!((tau_0, [])) |
for (@(L, p) <= matches) {                                // persistent for: process every
  announcement
  fire_L1!((tau_0, p), @"fired") |
  for (_ <- @"fired") { s0!((tau_0, [])) }
}
}

```

6.4 Running it

On the subject term $t = f(f(a, a), a)$ the program proceeds as follows.

- The driver builds the term channel τ_0 for t and sends $(\tau_0, [])$ on σ_{s_0} .
- σ_{s_0} inspects the head at position ϵ , finds f , and emits two parallel sends: $\sigma_{s_1}(\tau_0, [1])$ and $\sigma_{s_2}(\tau_0, [2])$.
- $\sigma_{s_1}(\tau_0, [1])$ inspects the head at position 1, finds f , announces $(L_1, [])$, and emits two further parallel sends.
- $\sigma_{s_2}(\tau_0, [2])$ inspects the head at position 2, finds a , and terminates (no a -arm in any state contract).
- The announcement $(L_1, [])$ is forwarded by the driver to **fire_L1**, which extracts σ , builds the RHS, splices it into τ_0 , and restarts the automaton.

Each function symbol of t is inspected exactly once per automaton run, and the two parallel sends from σ_{s_0} are scheduled by the Rholang runtime in either order or simultaneously.

6.5 The rho COMM rule: full compilation pipeline

We now revisit the rho-calculus COMM rule of §2.2, filling in the compilation pipeline downstream of the set automaton. The linearisation ℓ_{COMM}^b and the sharing constraint $\kappa_{\text{COMM}} = \{(1.1, 2.1)\}$ were derived there; we take them as given here and trace what happens between the matcher's announcement and the firing of the redex.

At announcement, the state-contract template of §4.4 emits

$$\text{matches}!(\ulcorner \text{COMM} \urcorner, p, \ulcorner \{(p.1.1, p.2.1)\} \urcorner).$$

The driver (§4.7) forwards the triple to the fire contract **fire_COMM**, which:

1. invokes **checkConstraint** on the pair $(p.1.1, p.2.1)$, which fires two parallel **traverse** calls returning sub-channels c_1, c_2 of the subject term, then asks the Rholang runtime whether $c_1 \equiv c_2$ as quoted processes;
2. if the check fails, aborts the firing and yields control back to the driver, which moves on to the next announcement;
3. if the check succeeds, calls **extractSubst** on $(\tau, p, \ulcorner \text{COMM} \urcorner)$ to extract a substitution σ mapping the metavariables of the original (unlinearised) LHS to subject-term channels — here, x to c_1 (equivalently c_2), B to the body channel at $p.1.2$, and Q to the payload channel at $p.2.2$;

4. calls `buildRhs` on the static RHS data of the COMM rule and σ , which produces a fresh term channel for the process $B[@Q/y]$ by composing the constructor contracts of the rho-calculus signature; the binder y in the RHS is handled by the bound-body protocol of §3.1;
5. calls `splice` on $(\tau, p, \text{new channel})$, which replaces the sub-channel of τ at p by the new channel and signals completion on `done`.

The driver receives the completion signal and restarts the automaton on the updated subject term (or, with a more refined construction, restarts from a partial-evaluation state determined by the rewritten position). The set automaton itself never inspects whether the two channels are equal; that information lives entirely in the fire stage, where it is decidable by the host structural-congruence check. Likewise, the host runtime is responsible for choosing an interleaving of the parallel `traverse`-calls inside `checkConstraint`, the parallel sends to successor states, and the parallel fire-contract invocations on concurrent announcements.

7 Discussion

7.1 Garbage collection

Term channels and state-channel messages are reference-counted naturally by Rholang’s structural congruence: a name with no pending sends and no pending receivers is structurally equivalent to $\mathbf{0}$ and is collected by the runtime. Our encoding does not introduce reference cycles (constructor contracts are leaves; state contracts are persistent but fixed in number; fire contracts are linear and consume themselves), so the standard rho-calculus garbage collector applies.

7.2 Dynamic rule sets

The compiler as stated treats the rule set as static. Adding rules at runtime — adding new state-contract code — is straightforward in Rholang: the new state contracts can be emitted as additional `contract` declarations on existing channels. The set automaton, however, must be rebuilt: adding a new pattern can change the dependency partition at every reachable state. A practical implementation would recompute the affected states only, but a full discussion is beyond this paper.

7.3 Equations, confluence, and FVP

The compiler’s correctness rests on two assumptions about E : convergence of $E^\downarrow$ (so that canonical forms exist and are unique) and the finite variant property (so that E -matching against the LHSs of $\mathcal{R}$ is decidable and reduces to syntactic matching after normalisation). When convergence fails — E is non-orientable, or KB completion does not terminate — the compiler emits a warning and falls back to an interleaved E -and- $\mathcal{R}$ rewriting strategy that is sound on confluent inputs but offers no completeness guarantee. When convergence holds but FVP fails — for example, equations like $f(f(x)) = f(x)$ in unrestricted form, which generate infinitely many variants — E -matching is undecidable in general, and the compiler can no longer guarantee that the set automaton, built on E -canonical LHSs, finds every E -equivalent redex. Sound but incomplete behaviour can be recovered by restricting to syntactic matching against canonical forms; complete behaviour requires either FVP, one of the other syntactic conditions listed in the footnote to Definition 2.2, or a manually provided E -matching oracle.

7.4 Comparison to direct compilation

A naive compiler from a GSLT presentation to Rholang would emit, for each rule $L_j \rightsquigarrow R_j$, a single `for` that pattern-matches L_j on the entire subject term. Such a compiler trades program size for redundant inspection: each function symbol of the subject term is inspected once per rule per ancestor position. The set-automaton route amortises this cost across the rule set, inspecting each function symbol once *total* per automaton run, at the price of a richer state space. For pattern sets of even modest size the saving is substantial, and the parallelism gained is, as we have shown, free.

8 Conclusion

The algorithm `compile` packages three independent strands of work — finitely presented graph-structured lambda theories, the Erkens–Groote set automaton, and the rho calculus — into a single compilation pipeline. The output is a Rholang interpreter that exposes every parallelism opportunity of the set automaton through Rholang’s asynchronous, name-passing semantics, without scheduling glue. We have shown the compiler is total, polynomial in the size of the automaton, and parallelism-preserving by construction. The pipeline gives a concrete answer to the question of how to implement, in a concurrent process calculus, the small-step semantics of any finitely presented GSLT.

Future work. Several directions remain. Incremental rebuild of the set automaton under dynamic rule changes would make the pipeline suitable for language-development environments. Partial evaluation of the automaton against statically known sub-contexts of the subject term would shrink the automaton — and the compiled program — in common cases such as pre-compiled standard libraries. Finally, a formal mechanisation of the parallelism-preservation theorem in a proof assistant would lend the construction the same kind of foundational credibility that the set-automaton construction enjoys in [1].

References

- [1] R. Erkens and J. F. Groote. A set automaton to locate all pattern matches in a term. *arXiv preprint 2106.15311*, 2021.
- [2] L. G. Meredith and M. Radestock. A reflective higher-order calculus. In *Electronic Notes in Theoretical Computer Science*, volume 141(5), pages 49–67, 2005.
- [3] S. Escobar, J. Meseguer, and R. Sasse. Effectively checking the finite variant property. In *Rewriting Techniques and Applications (RTA)*, LNCS 5117, pages 79–93, 2008.
- [4] S. Escobar, R. Sasse, and J. Meseguer. Folding variant narrowing and optimal variant termination. *Journal of Logic and Algebraic Programming*, 81(7–8), pages 898–928, 2012.
- [5] T. Trimble. Multisorted Lawvere theories. nLab entry, revision of 2024. <https://ncatlab.org/toddtrimble/published/multisorted+Lawvere+theories>.